

Mini Survivors — Programming Curriculum (Python)

A beginner curriculum for someone who has **never written a line of code**. You build the game together from the starter file, one small piece at a time.

- This is the **Python / pygame** track. There's a matching **JavaScript** version (`Curriculum-JavaScript.pdf`) that builds the exact same game in a browser.
- **New to code? Start with** `primer.py` — a 15-minute warm-up that prints the Python building blocks the game uses (variables, lists, `if`, loops, functions). No game window, just "type → run → see."
- Then build from `starter.py` (just a blue dot + the stages marked out).
- `game.py` is the finished **answer key** — peek only when stuck.
- **Run it after every change.** From the `python/` folder: `venv/bin/python starter.py` One-time setup first (also from the `python/` folder): `python3 -m venv venv venv/bin/pip install -r requirements.txt`

The code is laid out in **build order: Stage 1 at the top, Stage 7 at the bottom**, so you mostly work straight down the file. The `draw()` function and the game **loop** at the bottom grow as you go — each stage you uncomment one line in them.

The 5 golden rules of teaching this

1. **Run after every single change.** Even one line. "Change → run → look" is the heartbeat of the whole thing.
 2. **Let her type, even when it's slower.** Her hands on the keyboard, her typos, her fixing them — that's the learning.
 3. **Break it on purpose.** Change a `3` to `300`, delete a line, see what happens. Errors are how you discover what each piece was doing.
 4. **One new idea per stage.** Don't stack concepts.
 5. **Keep the run command on screen.** Closing and re-running the game every minute is normal and good — that loop of seeing the change is the whole point.
-

A few Python things to know up front

These are the only spots where Python looks different from the JavaScript track. Mention them when they come up; don't front-load them.

- **No { } for blocks.** Python uses a colon `:` and **indentation** (4 spaces) to show what's inside a function, loop, or `if`.
- **A "bag of values" is a dictionary:** `player = {"x": 5, "y": 9}`, and you read it with `player["x"]`.
- **Lists** use `.append(x)` to add and `.remove(x)` to delete.
- **global**: to *change* a top-level variable (like `score`) from inside a function, the function must say `global score` first. Reading it doesn't need that — only changing it.
- **No drawing helper needed:** pygame already has `pygame.draw.circle(...)`.

Stage 0 — "What even is code?" (15 min)

Goal: Demystify. Code is a list of instructions the computer follows top to bottom, very fast and very literally.

Do: Run `starter.py` (you'll see a blue dot in a window). Open the file in a text editor. Find this line and change the words in quotes:

```
pygame.display.set_caption("Mini Survivors")
```

Save, run it again. **The window's title changed because you changed the file.**

The aha: *I edit the file, I re-run, the program changes. That's programming.*

Stage 1 — Variables & drawing a dot (already done for you)

New concept: Variables (named boxes holding a value) and the screen as an x/y grid. Top-left is (0,0); x goes right, y goes **down**.

The starter already contains Stage 1 — read it together so it isn't magic. It's the **player** (a bag of numbers) and a **draw()** function that puts it on screen:

```
# the player is just a bag of numbers
player = {"x": W / 2, "y": H / 2, "size": 12, "speed": 3}

# draw the world (just the player for now)
def draw():
    screen.fill(BG)                                # wipe the screen
    pygame.draw.circle(screen, BLUE,
                        (int(player["x"]), int(player["y"])), player["size"])
```

Try it: make the dot bigger, move it to a corner, change `BLUE` to a different `(R, G, B)` value at the top.

The aha: A "thing" on screen is just a few numbers I picked.

Note vs JavaScript: no `circle()` helper — pygame gives us `pygame.draw.circle` for free.

Stage 2 — The game loop & moving (45 min)

New concept: The **game loop** — `update → draw`, ~120x/second, forever. Movement is just "change x a little, every frame." The `while running:` loop at the bottom **is** that loop.

In the **Stage 2 area**, add `move_player()`:

```
# move the player based on which keys are held
def move_player():
    keys = pygame.key.get_pressed() # which keys are down right now
    if keys[pygame.K_w] or keys[pygame.K_UP]: player["y"] -= player["speed"]
    if keys[pygame.K_s] or keys[pygame.K_DOWN]: player["y"] += player["speed"]
    if keys[pygame.K_a] or keys[pygame.K_LEFT]: player["x"] -= player["speed"]
    if keys[pygame.K_d] or keys[pygame.K_RIGHT]: player["x"] += player["speed"]

    # keep the player inside the screen
    player["x"] = max(player["size"], min(W - player["size"], player["x"]))
    player["y"] = max(player["size"], min(H - player["size"], player["y"]))
```

↪ In the loop, uncomment:

```
move_player() # Stage 2
```

Try it: change `speed`. Make a key snap you back to the middle.

The aha: Motion is just a number changing a tiny bit, many times a second.

Note vs JavaScript: no keyboard listeners to set up — pygame lets us ask "which keys are held?" every frame with `pygame.key.get_pressed()`.

Stage 3 — Lists & spawning enemies (45 min)

New concept: **Lists** (hold *many* things) and a little randomness.

At the top of the file, **uncomment** `import random`. Then in the **Stage 3 area**, add the enemies list + frame counter and the spawner:

```

# a list to hold all enemies, and a clock
enemies = []
frame = 0

# every so often, add an enemy at a random edge
def spawn_enemy():
    spawn_every = max(6, 45 - frame / 120)    # speeds up over time
    if frame % int(spawn_every) != 0:
        return

    if random.random() < 0.5:                  # left or right edge
        x = 0 if random.random() < 0.5 else W
        y = random.random() * H
    else:                                      # top or bottom edge
        x = random.random() * W
        y = 0 if random.random() < 0.5 else H

    enemies.append({"x": x, "y": y, "size": 10, "speed": 1.6})

```

↪ In `draw()`, **uncomment** the enemies block, and **in the loop**, **uncomment** the two Stage-3 lines:

```

for e in enemies:
    pygame.draw.circle(screen, RED, (int(e["x"]), int(e["y"])), e["size"])

```

```

frame += 1    # Stage 3
spawn_enemy() # Stage 3

```

Try it: spawn faster/slower (change the numbers), make enemies bigger.

The aha: I don't make 50 enemy variables. I make one list of 50 things.

Stage 4 — Loops & chasing (45 min)

New concept: `for` loops — "do the same thing to every item in a list." Also a first helper function: **distance** between two points.

At the top of the file, **uncomment** `import math`. Then **in the Stage 4 area**, add the distance helper and enemy movement:

```
# how far apart are two points? (straight from math class)
def distance(ax, ay, bx, by):
    return math.hypot(ax - bx, ay - by)

# move every enemy one step toward the player
def move_enemies():
    for e in enemies:
        d = distance(e["x"], e["y"], player["x"], player["y"])
        if d == 0:
            continue # never divide by zero
        e["x"] += ((player["x"] - e["x"]) / d) * e["speed"] # point it at you
        e["y"] += ((player["y"] - e["y"]) / d) * e["speed"]
```

↩ In the loop, uncomment:

```
move_enemies() # Stage 4
```

Try it: change enemy `speed` . Flip a `+` to a `-` so they flee (funny, and instructive).

The aha: One loop handles 1 enemy or 1,000. I wrote the rule once.

Stage 5 — Functions & auto-shooting (45 min)

New concept: Functions — a named box of instructions, so the program reads like a to-do list. (You've been writing them already; now name the idea.)

First, ↩ go back to the **Stage 1 player** and add a fire rate:

```
"fireRate": 30, # shoot every 30 frames. Lower = faster.
```

In the **Stage 5 area**, add the bullets list and the shooter:

```

# a list to hold bullets
bullets = []

# every fireRate frames, fire a bullet at the nearest enemy
def auto_shoot():
    if frame % player["fireRate"] != 0: # not time yet
        return
    if len(enemies) == 0: # nothing to shoot
        return

    # find the nearest enemy
    target = enemies[0]
    best = float("inf")
    for e in enemies:
        d = distance(player["x"], player["y"], e["x"], e["y"])
        if d < best:
            best = d
            target = e

    # fire a bullet aimed at it
    d = distance(player["x"], player["y"], target["x"], target["y"])
    bullets.append({
        "x": player["x"], "y": player["y"],
        "dx": ((target["x"] - player["x"]) / d) * 6, # 6 = bullet speed
        "dy": ((target["y"] - player["y"]) / d) * 6,
        "size": 4,
    })

```

↩ In `draw()`, uncomment the bullets block, and in the loop, uncomment `auto_shoot()`:

```

for b in bullets:
    pygame.draw.circle(screen, YELLOW, (int(b["x"]), int(b["y"])), b["size"])

```

```

auto_shoot() # Stage 5

```

(Bullets won't move yet — that's the next stage.)

Try it: shoot faster (`fireRate`), bigger bullets, fire two at once.

The aha: The loop reads like a to-do list: move, spawn, chase, shoot.

Stage 6 — Conditionals & collision (45 min)

New concept: `if` statements (decisions) and **collision** — two circles touch when the distance between them is less than their sizes added up. Also your first taste of `global`.

In the **Stage 6** area, add the score + game-over flag, and `move_bullets()`. Note `global score` — the function *changes* score, so it must declare it:

```
# score, and a flag for whether the game has ended
score = 0
game_over = False

# move bullets, delete off-screen ones, kill enemies on contact
def move_bullets():
    global score
    for b in bullets[:]:          # the[:] copy lets us remove safely
        b["x"] += b["dx"]
        b["y"] += b["dy"]

        if b["x"] < 0 or b["x"] > W or b["y"] < 0 or b["y"] > H:  # off screen?
            bullets.remove(b)
            continue

        for e in enemies[:]:
            if distance(b["x"], b["y"], e["x"], e["y"]) < b["size"] + e["size"]:
                score += 10
                bullets.remove(b)    # remove the bullet
                enemies.remove(e)    # remove the enemy
                break
```

↩ Back up in `move_enemies()`, add `global game_over` as its first line, and one line inside the `for` loop so touching the player ends the game:

```
def move_enemies():
    global game_over
    for e in enemies:
        ...
        if d < e["size"] + player["size"]:    # touching the player?
            game_over = True
```

↩ In the loop, uncomment:

```
move_bullets()    # Stage 6
```

Nothing happens *visually* when `game_over` becomes true yet — the next stage adds the game-over screen. For now, watch your bullets delete enemies.

Try it: make the player's `size` tiny (hard) or huge (easy). Give enemies `"hp": 2` and subtract 1 per hit — you're reinventing a feature!

The aha: *The whole game is just "if this is true, do that," repeated.*

Stage 7 — The HUD, game-over screen & restart (45 min)

New concept: showing the game's **state** on screen, and **restarting**. This is the stage where you *wrap* the loop — the one time we restructure it. (The fonts you need are already in setup.)

In `draw()`, add the HUD at the bottom:

```
screen.blit(font.render("Time: " + str(frame // FPS) + "s", True, WHITE), (10, 10))
screen.blit(font.render("Score: " + str(score), True, WHITE), (10, 30))
```

In the **Stage 7** area, add the game-over screen and a reset function:

```
# the game-over screen
def draw_game_over():
    t1 = big_font.render("GAME OVER", True, WHITE)
    t2 = font.render("You survived " + str(frame // FPS) + " seconds", True, WHITE)
    t3 = font.render("Final score: " + str(score), True, WHITE)
    t4 = font.render("Press R to play again", True, WHITE)
    screen.blit(t1, (W / 2 - t1.get_width() / 2, H / 2 - 40))
    screen.blit(t2, (W / 2 - t2.get_width() / 2, H / 2 + 5))
    screen.blit(t3, (W / 2 - t3.get_width() / 2, H / 2 + 30))
    screen.blit(t4, (W / 2 - t4.get_width() / 2, H / 2 + 60))

# reset everything for a new run
def reset_game():
    global enemies, bullets, frame, score, game_over
    enemies = []
    bullets = []
    frame = 0
    score = 0
    game_over = False
    player["x"] = W / 2
    player["y"] = H / 2
```

Finally, **wire it into the loop**: handle the R key in the event section, and **wrap the update + draw** so the game-over screen shows when you're dead:


```

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
    if event.type == pygame.KEYDOWN and event.key == pygame.K_r and game_over:
        reset_game()

if game_over:
    draw_game_over()
else:
    move_player()      # Stage 2
    frame += 1         # Stage 3
    spawn_enemy()      # Stage 3
    move_enemies()     # Stage 4
    auto_shoot()       # Stage 5
    move_bullets()     # Stage 6
    draw()

```

That's the whole game. 🎮

Try it: worth more points per kill; add `score += 1` each frame to reward survival; speed enemies up over time.

The aha: Most "features" are the same recipe: make a value, change it, show it.

Capstone — "Same game, another language" (30 min)

Open `game.py` next to `index.html`. They share the same **Stage** labels. Skim them together.

The ideas — variables, loops, lists, functions, ifs — are **identical**. Only the spelling changes:

Idea	Python	JavaScript
Bag of values	<code>player["x"]</code>	<code>player.x</code>
Add to a list	<code>enemies.append(x)</code>	<code>enemies.push(x)</code>
Loop a list	<code>for e in enemies:</code>	<code>for (let e of enemies)</code>
Block of code	<code>:</code> + indentation	<code>{ ... }</code>
The game loop	<code>while running:</code>	<code>requestAnimationFrame(loop)</code>
Square root	<code>math.hypot(...)</code>	<code>Math.sqrt(...)</code>

The point to land: you didn't learn Python — you learned to program, and that transfers to any language.

The browser version needs nothing to run; Python needs a one-time `pip install pygame` (handled by the `venv`). That's the main practical difference — the *thinking* is the same.

Where to go next (pick what excites her)

- **More weapons:** spread shot, a slow huge bullet, an orbiting shield.
- **Enemy variety:** fast-weak vs slow-tanky (bring back `hp`).
- **Power-ups:** the green XP gems again — but as a thing *she* designs.
- **Juice:** screen shake, particles on kill, sound. Small effort, huge payoff.
- **Save the high score** to a file — first taste of data that outlives the program.

The goal was never to finish a game. It's to make her believe **she can make the computer do what she wants**. Once that clicks, she'll lead.